

**UNIVERSIDADE FEDERAL DO MARANHÃO**

Centro de Ciências Exatas e Tecnologia
Departamento de Informática — DEINF

**RELATÓRIO PRÁTICO 1: API UNIX E
CONCORRÊNCIA**

Coordenação de processos com `fork()`, `pipe()` e `wait()`
no programa `bot_search`

Luana Lorhanni Pacheco Santos (2022036611)

Melissa Rodrigues Palhano (20240065535)

São Luís - MA
Outubro de 2026

1. Resumo

Este relatório descreve o projeto, a implementação e a verificação de `bot_search`, um coordenador mestre-escravo no modelo Bag-of-Tasks. O processo pai recebe um termo e uma lista de arquivos de texto. Para cada arquivo ele abre um pipe anônimo e cria, com `fork()`, um filho cuja única tarefa é contar as ocorrências do termo naquele arquivo. O filho devolve um inteiro pela ponta de escrita do seu pipe. O pai lê cada resposta, recolhe os filhos com `wait(NULL)` e imprime o relatório por arquivo e o total.

Bag-of-Tasks, neste problema, é um conjunto de tarefas independentes e da mesma natureza: cada tarefa é o par (termo, arquivo). Não há dependência de dados entre arquivos, não há ordem obrigatória de execução e o resultado global é a soma dos resultados parciais. Esse formato admite um mestre que distribui e agrega, e trabalhadores que não conversam entre si.

A distribuição acontece no momento do `fork()`. O filho herda o termo, o nome do arquivo — ambos em `argv` — e os descritores abertos. A partir daí o filho não recebe mais nenhuma mensagem. O canal serve só para a volta do resultado. Essa escolha cabe no enunciado: o pai já conhece o caminho do arquivo antes de criar o filho, então não precisa de um segundo pipe para enviar trabalho.

O paralelismo é o dos processos, escalonados pelo núcleo. O pai não implementa uma fila, não limita o número de filhos ativos e não reparte um arquivo entre vários filhos. Com N arquivos há N processos simultâneos na fase de contagem. A agregação é serial e acontece no pai, depois que cada inteiro está disponível no pipe correspondente.

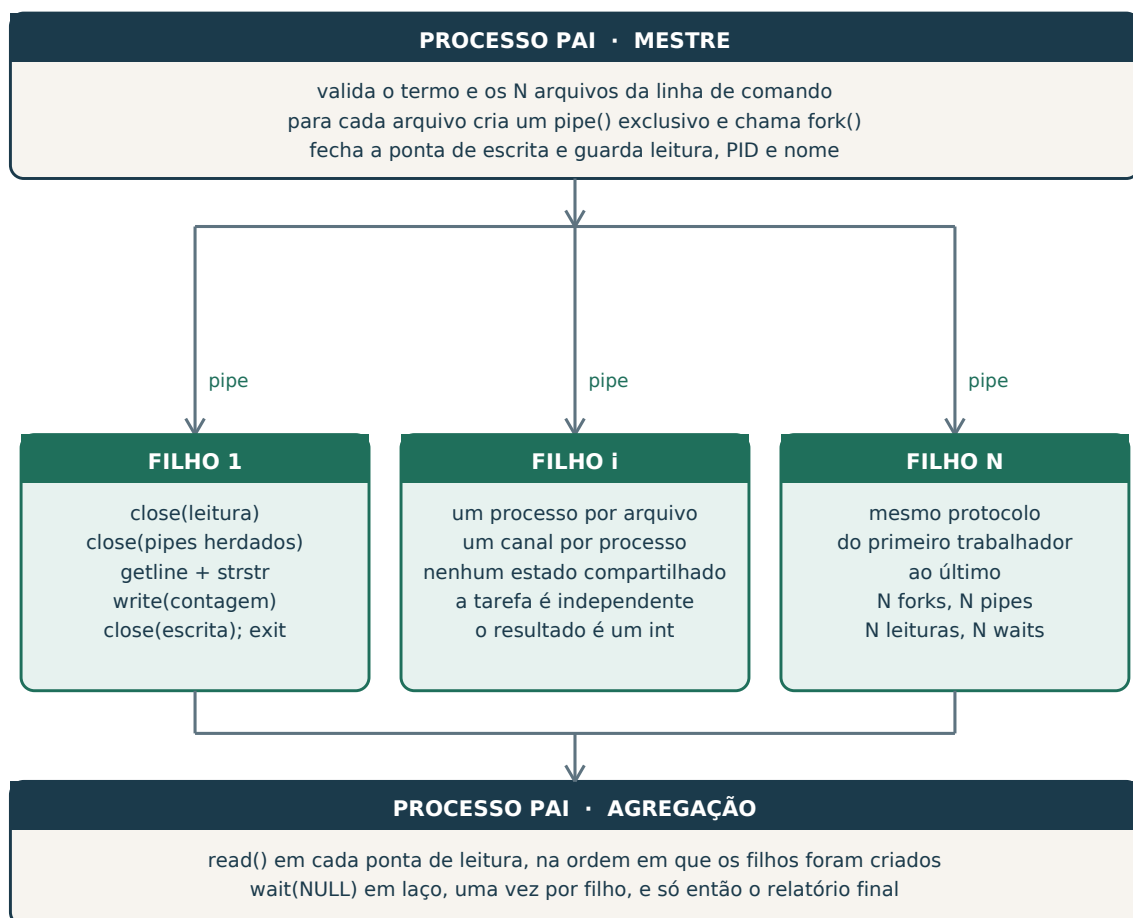


Figura 1. Topologia do `bot_search`. Cada filho possui um pipe exclusivo; o pai é o único leitor.

2. Problema e requisitos

O enunciado fixa a arquitetura. Há um mestre e exatamente N escravos, N sendo o número de caminhos passados depois do termo. A comunicação é unidirecional, do filho para o pai, por um canal criado com `pipe()` antes do `fork()`. O filho conta o termo com `strstr()` ou equivalente, escreve o total com `write()`, fecha a ponta de escrita e termina. O pai fecha a ponta de escrita, lê com `read()`, espera os N filhos e publica o consolidado.

A invocação é `./bot_search <termo> <arquivo1> ... <arquivoN>`. O formato de saída segue o exemplo do enunciado, inclusive a linha “TOTAL CONSOLIDADOS”, reproduzida como está no roteiro.

3. Protocolo do pipe

A mensagem é um `int` no formato nativo do processo, escrito com `write(fd, &ocorrencias, sizeof(int))`. Pai e filho são o mesmo programa depois do `fork`, na mesma máquina, então compartilham tamanho, alinhamento e endiandade. Não há parsing de texto. Uma contagem válida é sempre maior ou igual a zero. O valor `-1` é reservado para falha de abertura ou de leitura do arquivo e nunca é somado (IEEE; THE OPEN GROUP, 2018).

Na execução com os três exemplos, o `strace` mostra três escritas de exatamente 4 bytes nos pipes. Os payloads observados foram `\5\0\0\0`, `\3\0\0\0`, `\3\0\0\0`. Em little-endian, `\3\0\0\0` é o inteiro 3 e `\5\0\0\0` é o inteiro 5, que são as contagens de `doc1.txt`, `doc2.txt` e `doc3.txt`. O pai só aceita a resposta quando `read()` devolve os 4 bytes. Um fechamento sem escrita aparece como leitura curta e é reportado como resposta inválida.

Quatro bytes estão muito abaixo de `PIPE_BUF` (4 KiB no POSIX; o buffer do pipe no Linux é bem maior, tipicamente 64 KiB). A escrita desse tamanho é atômica: o pai não observa um inteiro rasgado no meio. O filho também não enche o buffer, então não bloqueia em `write()` à espera de o pai ler. O deadlock clássico do pipe — escritor bloqueado porque o leitor não consome, leitor bloqueado porque ainda segura a ponta de escrita e espera fim de arquivo — não se forma. O protocolo não usa fim de arquivo. Cada lado fecha a ponta que não usa, e a mensagem tem tamanho conhecido (STEVENS; RAGO, 2013).

4. Ciclo de vida dos descritores

Depois do `fork()`, os dois processos enxergam as mesmas pontas. Se o pai esquecesse a ponta de escrita aberta, uma leitura baseada em fim de arquivo nunca terminaria. Se um filho esquecesse a ponta de leitura de um irmão, aquele pipe teria um leitor extra até o processo terminar. O código fecha as pontas na hora em que o papel de cada processo fica definido.

Momento	Pai	Filho recém-criado
<code>pipe()</code> retornou	Possui a ponta de leitura e a de escrita do canal novo.	Ainda não existe.
<code>fork()</code> retornou	Continua com as duas pontas do canal novo e com as leituras dos canais anteriores.	Herda as duas pontas do próprio canal e as leituras dos irmãos já criados. As escritas dos canais anteriores já foram fechadas pelo pai, então o filho não as herda.
Logo em seguida	close da escrita. Guarda a leitura, o PID e o caminho.	close da própria leitura e close da leitura de cada irmão anterior.
Fim da tarefa	Ainda não leu, ou já está bloqueado no <code>read</code> .	<code>write</code> do inteiro, close da escrita, <code>exit</code> . O <code>FILE*</code> do arquivo foi fechado com <code>fclose</code> .
Depois do <code>read</code>	close da leitura daquele filho.	Já encerrou. Entre o <code>exit</code> e o <code>wait</code> o processo fica zumbi, por um intervalo curto.
Laço final	<code>wait(NULL)</code> exatamente uma vez por filho criado. <code>free</code> do vetor de trabalhadores.	Recolhido. O núcleo libera o restante do espaço de endereçamento.

Há um detalhe de herança de memória que convém explicitar. O pai aloca com `malloc` o vetor que associa PID, descritor e caminho. O `fork()` duplica esse bloco no espaço do filho (de fato, as páginas ficam compartilhadas em cópia-na-escrita até alguém escrever). O filho usa o vetor só para fechar os descritores herdados e em seguida libera a própria cópia. Essa liberação não devolve o bloco do pai: são dois espaços de endereçamento. O pai libera o dele depois do `wait`. Sem o `free` no filho, o Valgrind com `--trace-children` classificaria o bloco herdado como ainda alcançável, porque `exit()` não volta ao `main` para liberá-lo. O núcleo, de todo modo, recolhe a memória no término; a liberação explícita torna o rastreo inequívoco.

O `printf` do cabeçalho é descarregado com `fflush(stdout)` antes de cada `fork()`. Com a saída num terminal, o `stdout` é bufferizado por linha e o problema quase não aparece. Com a saída redirecionada para um arquivo ou para um pipe, o buffer fica cheio de modo integral: o filho nasceria com uma cópia do texto ainda não escrito e o cabeçalho sairia duplicado. Na execução redirecionada dos três exemplos, a linha “Criando” aparece uma vez.

5. Sincronização

A ordem exigida pelo enunciado é: ler as respostas parciais e, em seguida, esperar os filhos. O `read()` no pipe bloqueia até que o filho escreva ou feche a ponta. Quando os quatro bytes chegam, o pai já tem o dado daquele trabalhador, mesmo que o processo filho ainda não tenha sido recolhido. O `wait(NULL)` vem depois, num laço de tamanho criados, e existe para eliminar zumbis e para o programa só declarar o término quando o núcleo confirmou o `exit` de todos.

O pai lê na ordem de criação, não na ordem em que os filhos terminam. A associação entre contagem, PID e nome de arquivo fica estável: ela mora na estrutura preenchida no `fork()`, não na ordem em que o escalonador roda os filhos. Um filho lento atrasa a impressão dos resultados seguintes, porque a leitura é serial. Não atrasa o trabalho deles. Os outros filhos seguem contando e podem deixar o inteiro pronto no buffer do kernel. Para uma mensagem de 4 bytes, essa espera é só de apresentação.

O enunciado pede `wait(NULL)`, que recolhe qualquer filho e descarta o status. Por isso o pai não descobre uma falha de arquivo olhando o código de saída do processo. A falha viaja no próprio inteiro. O status do filho continua coerente com o que aconteceu — `exit(0)` quando a contagem foi entregue, `exit(EXIT_FAILURE)` quando o arquivo não pôde ser lido ou o `write()` não entregou os 4 bytes — mas quem decide o código de saída do `bot_search` é o pai, com base no que leu nos pipes e em se todos os filhos chegaram a ser criados.

6. Contagem do termo

O enunciado manda buscar a palavra-chave com `strstr()` ou equivalente. A implementação segue esse contrato: a busca é por substring, distingue maiúsculas de minúsculas e avança o tamanho do termo a cada acerto, então ocorrências sobrepostas não são contadas duas vezes. Em “aaaa”, o termo “aa” vale 2. Em “processos”, o termo “processo” vale 1, porque os sete primeiros caracteres coincidem.

A leitura é linha a linha com `getline()`, que aloca e cresce o buffer. A última linha sem caractere de nova linha entra na contagem: o arquivo que contém só a palavra “processo”, sem `\n`, vale 1. Arquivo vazio vale 0, que é contagem, e se distingue do `-1` de erro. `ferror()` separa fim de arquivo de falha de leitura; nesse caso o filho também devolve `-1`.

O termo vazio é recusado antes de qualquer `fork()`. Com string vazia, `strstr()` encontra “ocorrência” em todo ponto, e o laço não andaria. Essa checagem é uma borda do algoritmo, além da checagem de `argc < 3`.

7. Falhas

Cada chamada de sistema usada no caminho principal tem o retorno testado: pipe, fork, read, write, wait e malloc. fopen nulo produz perror no filho, com o nome do caminho, e o inteiro -1 no pipe. O pai imprime a falha daquele arquivo e segue para os outros. As tarefas são independentes: um arquivo ilegível não cancela a contagem dos demais.

Se pipe() ou fork() falha no meio do laço, o pai para de criar trabalhadores, fecha o canal que não chegou a gerar um filho e ainda lê e espera os que já existem. O processo termina com status de erro. O mesmo status de erro ocorre quando algum filho reporta arquivo ilegível ou quando a leitura do pipe não traz um inteiro completo. Termo ausente, com total zero, é sucesso: a busca ocorreu.

Para um único arquivo a frase do cabeçalho fica no singular (“1 trabalhador”) e o encerramento diz “O processo filho finalizou.” Com dois ou mais, o texto coincide com o exemplo do enunciado: “Criando N trabalhadores” e “Todos os N processos filhos finalizaram.”

8. Ambiente e como reproduzir

As medições deste relatório foram obtidas em Ubuntu 24.04.5 LTS, núcleo Linux 6.6.87.2-microsoft-standard-WSL2 x86_64, compilador GCC 13.3.0, com as flags do Makefile: `-Wall -Wextra -pedantic -std=c11 -g`. A compilação terminou sem diagnóstico. O diretório de trabalho foi `bot_search/`.

```
make
make run
./bot_search "processo" exemplos/doc1.txt exemplos/doc2.txt exemplos/doc3.txt
```

Os arquivos `exemplos/doc1.txt`, `doc2.txt` e `doc3.txt` acompanham o código. O segundo conjunto, usado na busca por “SO”, foi criado só para o ensaio e o conteúdo integral está na seção 9, de modo que a conta pode ser refeita à mão.

9. Resultados

9.1 Três arquivos e o termo “processo”

`doc1.txt` contém três linhas e a palavra em cada uma delas. `doc2.txt` contém “processo” duas vezes na primeira linha e uma vez na segunda. `doc3.txt` contém “processos”, que contribui com uma ocorrência de substring, e a frase “Um processo escreve e outro processo lê, processo por processo.”, que contribui com quatro. A terceira linha de `doc3.txt`, sobre threads, contribui com zero. A soma é $3 + 3 + 5 = 11$. O `grep -o processo` devolve os mesmos três números.

```
melissa trabalho-so-bag-of-tasks (develop)
> ./bot_search "processo" exemplos/doc1.txt exemplos/doc2.txt exemplos/doc3.txt
[Mestre PID: 2108] Criando 3 trabalhadores Bag-of-Tasks...
[Mestre] Resposta do Filho (PID 2109 - 'exemplos/doc1.txt'): 3 ocorrências.
[Mestre] Resposta do Filho (PID 2110 - 'exemplos/doc2.txt'): 3 ocorrências.
[Mestre] Resposta do Filho (PID 2111 - 'exemplos/doc3.txt'): 5 ocorrências.
[Mestre] Todos os 3 processos filhos finalizaram.
-----
TOTAL CONSOLIDADOS: 11 ocorrências do termo 'processo'.
```

Figura 2. Execução com os três arquivos de exemplo. PIDs variam a cada rodada; as contagens não.

9.2 Outro conjunto de arquivos, termo “SO”

O segundo conjunto tem o conteúdo abaixo. A conta manual de “SO” é 3, 1 e 0.

```
aula1.txt
A disciplina de SO estuda processos.
SO também estuda SO.
```

```
aula2.txt
Pipes e fork aparecem em SO.
```

```
aula3.txt
Nada de sistemas aqui.
Outra linha.
```

Na primeira linha de `aula1.txt` há um “SO”. Na segunda há dois. `aula2.txt` tem um. `aula3.txt` não tem o termo, e o programa imprime 0 ocorrências, que é resultado válido. O total é 4.

```
melissa trabalho-so-bag-of-tasks (develop)
> ./bot_search "S0" /tmp/bot_search_aulas/aula1.txt /tmp/bot_search_aulas/aula2.txt /tmp/bot_search_aulas/aula3.txt
[Mestre PID: 2112] Criando 3 trabalhadores Bag-of-Tasks...
[Mestre] Resposta do Filho (PID 2113 - '/tmp/bot_search_aulas/aula1.txt'): 3 ocorrências.
[Mestre] Resposta do Filho (PID 2114 - '/tmp/bot_search_aulas/aula2.txt'): 1 ocorrências.
[Mestre] Resposta do Filho (PID 2115 - '/tmp/bot_search_aulas/aula3.txt'): 0 ocorrências.
[Mestre] Todos os 3 processos filhos finalizaram.
-----
TOTAL CONSOLIDADOS: 4 ocorrências do termo 'S0'.
```

Figura 3. Mesmo programa, outros três arquivos e outro termo. O total 4 confere com a leitura manual.

9.3 Arquivo ilegível no meio de uma leva válida

Com um arquivo existente e um caminho inexistente, o filho do arquivo válido devolve 3 e o filho do caminho ausente escreve -1. O stderr do segundo filho traz “No such file or directory”. O total fica em 3, e o processo pai termina com código 1. Diretório no lugar de arquivo e arquivo com modo 000 foram ensaiados à parte: ambos produzem perror no filho (“Is a directory” e “Permission denied”), linha de erro no relatório e código de saída 1.

```
melissa trabalho-so-bag-of-tasks (develop)
> ./bot_search "processo" exemplos/doc1.txt /tmp/bot_search_aulas/naoexiste.txt
[Mestre PID: 2116] Criando 2 trabalhadores Bag-of-Tasks...
/tmp/bot_search_aulas/naoexiste.txt: No such file or directory
[Mestre] Resposta do Filho (PID 2117 - 'exemplos/doc1.txt'): 3 ocorrências.
[Mestre] Resposta do Filho (PID 2118 - '/tmp/bot_search_aulas/naoexiste.txt'): erro ao ler o arquivo.
[Mestre] Todos os 2 processos filhos finalizaram.
-----
TOTAL CONSOLIDADOS: 3 ocorrências do termo 'processo'.
```

Figura 4. Uma tarefa falha e a outra é consolidada. O código de saída do processo foi 1.

9.4 Maiúsculas, um único trabalhador e volume

O termo “Processo”, com P maiúsculo, percorre os três exemplos e totaliza 0. O mesmo ocorre com “kernel”, que não está nos textos. São buscas concluídas, com código de saída 0.

```
melissa trabalho-so-bag-of-tasks (develop)
> ./bot_search "Processo" exemplos/doc1.txt exemplos/doc2.txt exemplos/doc3.txt
[Mestre PID: 2121] Criando 3 trabalhadores Bag-of-Tasks...
[Mestre] Resposta do Filho (PID 2122 - 'exemplos/doc1.txt'): 0 ocorrências.
[Mestre] Resposta do Filho (PID 2123 - 'exemplos/doc2.txt'): 0 ocorrências.
[Mestre] Resposta do Filho (PID 2124 - 'exemplos/doc3.txt'): 0 ocorrências.
[Mestre] Todos os 3 processos filhos finalizaram.
-----
TOTAL CONSOLIDADOS: 0 ocorrências do termo 'Processo'.
```

Figura 5. A busca distingue maiúsculas. Total zero é sucesso da consulta.

```
melissa trabalho-so-bag-of-tasks (develop)
> ./bot_search "processo" exemplos/doc2.txt
[Mestre PID: 2129] Criando 1 trabalhador Bag-of-Tasks...
[Mestre] Resposta do Filho (PID 2130 - 'exemplos/doc2.txt'): 3 ocorrências.
[Mestre] 0 processo filho finalizou.
-----
TOTAL CONSOLIDADOS: 3 ocorrências do termo 'processo'.
```

Figura 6. Um único arquivo. O cabeçalho e o encerramento vão para o singular; doc2.txt contribui com 3.

Trinta cópias de doc1.txt geram trinta filhos e total 90, que é 30×3 . A figura mostra o começo e o fim dessa saída; as trinta linhas de resposta foram produzidas na execução (a saída completa tem 30 linhas de contagem). Um arquivo de 100 000 linhas, cada uma com duas vezes o termo, produz 200 000. Quatro ocorrências em duas linhas (“processo processo” e “processoprocesso”) produzem 4. “aaaa” procurando “aa” produz 2.

```

melissa trabalho-so-bag-of-tasks (develop)
> ./bot_search "processo" /tmp/bot_search_aulas/lote/f*.txt
[Mestre PID: 2141] Criando 30 trabalhadores Bag-of-Tasks...
[Mestre] Resposta do Filho (PID 2142 - '/tmp/bot_search_aulas/lote/f01.txt'): 3 ocorrências.
[Mestre] Resposta do Filho (PID 2143 - '/tmp/bot_search_aulas/lote/f02.txt'): 3 ocorrências.
[Mestre] Resposta do Filho (PID 2144 - '/tmp/bot_search_aulas/lote/f03.txt'): 3 ocorrências.
...
[Mestre] Resposta do Filho (PID 2171 - '/tmp/bot_search_aulas/lote/f30.txt'): 3 ocorrências.
[Mestre] Todos os 30 processos filhos finalizaram.
-----
TOTAL CONSOLIDADOS: 90 ocorrências do termo 'processo'.

```

Figura 7. Trecho da execução com 30 arquivos. O total medido na saída completa foi 90.

Ensaio	Saída	Total	Código
Sem argumentos	mensagem de uso	—	1
Termo vazio	recusa antes do fork	—	1
processo nos 3 exemplos	3 + 3 + 5	11	0
grep -o processo nos 3 exemplos	3, 3 e 5	11	—
SO em aula1, aula2, aula3	3 + 1 + 0	4	0
Processo nos 3 exemplos	termo ausente por causa da caixa	0	0
kernel nos 3 exemplos	termo ausente	0	0
doc1.txt e um caminho inexistente	3 e erro de leitura	3	1
diretório exemplos/	erro de leitura	0	1
arquivo modo 000	Permission denied	0	1
só doc2.txt	singular no relatório	3	0
duas linhas com quatro substrings	conta várias por linha	4	0
aaaa com termo aa	sem contar sobreposição	2	0
arquivo sem \n final	última linha entra	1	0
arquivo vazio	contagem zero	0	0
30 cópias de doc1.txt	30 filhos	90	0
100000 linhas × 2	volume	200000	0
stdout redirecionado, 3 exemplos	cabeçalho uma vez	11	0

Tabela 1. Ensaios executados para este relatório. Os totais foram lidos da saída do programa nesta rodada.

10. O que o strace e o Valgrind mostram

No rastreamento dos três exemplos, `strace -f` registrou 3 chamadas `pipe2` (é assim que o `pipe()` da `glibc` chega ao núcleo neste sistema), 3 `clone` com as flags de `fork`, e 3 `wait4`, que é o serviço por trás de `wait(NULL)`. Há uma escrita de 4 bytes por filho, com os inteiros 3, 3 e 5, e o pai só imprime o relatório depois dessas leituras. O número de canais, de processos e de esperas é exatamente N.

O Valgrind foi executado com `--leak-check=full`, `--show-leak-kinds=all`, `--track-fds=yes` e `--trace-children=yes`. Foram 4 processos (pai e três filhos). Em cada um, `definitely lost = 0` bytes, 3 descritores abertos no `exit` (`stdin`, `stdout` e `stderr`) e `ERROR SUMMARY = 0`. O pai e os filhos encerram com “All heap blocks were freed — no leaks are possible”.

```

melissa trabalho-so-bag-of-tasks (develop)
> valgrind --leak-check=full --show-leak-kinds=all --track-fds=yes --trace-children=yes ./bot_search "processo"
exemplos/doc1.txt exemplos/doc2.txt exemplos/doc3.txt
==2181== HEAP SUMMARY:
==2181==      in use at exit: 0 bytes in 0 blocks
==2181==    total heap usage: 2 allocs, 2 frees, 4,144 bytes allocated
==2181==
==2181== All heap blocks were freed -- no leaks are possible
==2181==
==2181== For lists of detected and suppressed errors, rerun with: -s
==2181== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

Figura 8. Trecho final do Valgrind nesta rodada, no último processo rastreado. O mesmo fecho aparece nos quatro.

O log bruto contém 12 linhas de resumo de descritores, vazamento e erro, todas coerentes com o quadro acima. Código de saída do Valgrind: 0.

11. Limites conscientes

A leitura do pai é bloqueante e segue a ordem dos arquivos. Se um filho nunca escrever e nunca fechar a ponta — por exemplo, se ficar preso dentro da contagem — o pai permanece no `read()` e ainda não executou o `wait` dos filhos que já terminaram. Esses ficam zumbis até o desbloqueio. O enunciado pede essa leitura explícita em cada descritor; para o tamanho de mensagem usado aqui, o caso normal não bloqueia o filho.

O número de trabalhadores é limitado pelos descritores do processo pai (`RLIMIT_NOFILE`) e pela disposição do núcleo em criar processos. Cada tarefa consome um pipe. O código trata a falha de `pipe()` ou `fork()` parando a criação e ainda recolhendo quem já nasceu. Não há um pool menor que `N`.

A contagem parcial cabe num `int`. O maior ensaio, 200 000, está longe de `INT_MAX`. Acima desse limite a soma em inteiro com sinal transborda, e isso é comportamento indefinido em C. O total do pai é um `long`, largo o bastante para somar as parciais que um `int` consegue representar.

`read()` e `write()` podem retornar de menos, ou falhar com `EINTR`, se um sinal interromper a chamada. O programa trata tamanho diferente de 4 bytes como falha de comunicação. Não há laço de retomada. Para essa carga, sem sinais na aplicação, a escrita de 4 bytes abaixo de `PIPE_BUF` completa de uma vez. Fica registrado como limite, não como falha observada.

A busca não isola palavra inteira. Quem procurar “processo” também encontra o prefixo de “processos”. Isso é o comportamento de `strstr()`, que é a função nomeada no enunciado, e está coberto pelo ensaio de `doc3.txt`.

12. Conclusão

O `bot_search` realiza o Bag-of-Tasks pedido com um processo e um pipe por arquivo, contagem no filho, inteiro de volta pelo canal e agregação no pai depois de `wait(NULL)`. A saída do caso principal acompanha o exemplo do roteiro. As contagens conferem com leitura manual e com `grep -o`. O rastreo de sistema mostra `N` canais, `N` criações e `N` esperas. O rastreo de recursos mostra descritores de pipe fechados e heap liberado no pai e nos filhos.

O desenho cabe no problema porque as tarefas não compartilham estado: o único dado que cruza a fronteira do processo é um inteiro de quatro bytes, num canal que deixa de ter pontas sobrando assim que cada processo assume o seu papel.

Referências

IEEE; THE OPEN GROUP. **IEEE Std 1003.1-2017**: IEEE standard for information technology: portable operating system interface (POSIX): base specifications, issue 7. New York: IEEE, 2018.

STEVENS, W. Richard; RAGO, Stephen A. **Advanced programming in the UNIX environment**. 3. ed. Upper Saddle River: Addison-Wesley, 2013.